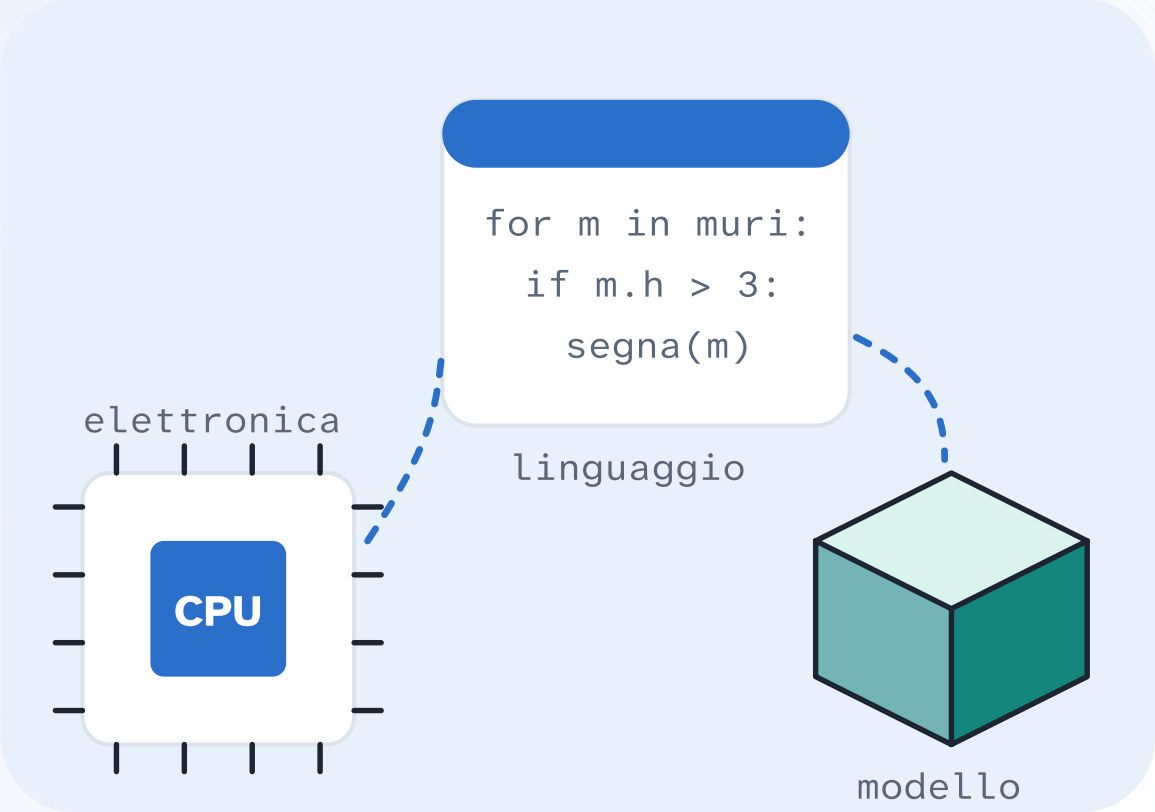
I MANUALI DI BIM NEWS · NELLO STUDIO

Linguaggi di programmazione per chi lavora nel BIM

Che cosa sono i linguaggi, come si riconoscono le loro famiglie, quali si incontrano in uno studio di progettazione e che cosa succede davvero, dentro il computer, tra la pressione di un tasto e la modifica di un modello.

Per chi ha già visto un po' di codice e vuole orientarsi, senza essere un programmatore.

16 capitoli | oltre 60 linguaggi | esempi in Python, C#, Dynamo, SQL | Edizione novembre 2026



Come usare questa guida

Per leggere codice non serve conoscere sessanta linguaggi: serve riconoscere a quale **famiglia** appartiene un linguaggio, perché le famiglie condividono idee, parole e punteggiatura. Chi sa leggere un po' di Python riconosce la struttura di Ruby; chi ha visto C# legge Java quasi senza accorgersene.

La guida segue un percorso dal basso verso l'alto. I primi quattro capitoli spiegano le idee di fondo e che cosa succede dentro il computer, con un'attenzione particolare a due temi che chi lavora nel BIM incontra ogni giorno: i numeri decimali delle coordinate e gli identificativi univoci degli oggetti. Dal capitolo 5 si passa ai linguaggi, famiglia per famiglia, con tabelle che dicono dove si usano, che cosa fanno bene e dove fanno fatica. Il capitolo 11 raccoglie quelli del mondo AEC e BIM, con esempi pronti da leggere.

LE ANALOGIE DA CANTIERE

Lungo la guida trovi riquadri come questo: collegano un concetto informatico a qualcosa che chi progetta conosce già, dalle famiglie di Revit al capitolato, dal cronoprogramma al computo.

LE DATE E LE VERSIONI

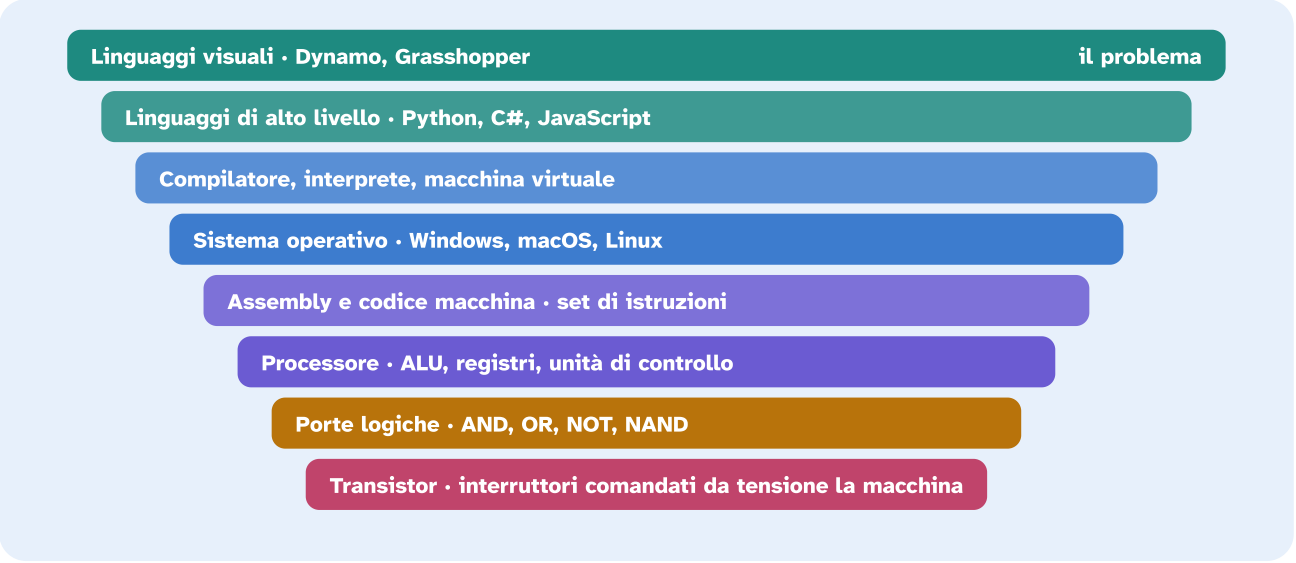
Gli anni di nascita dei linguaggi sono un riferimento storico. Software, API e versioni dei linguaggi cambiano spesso: quando lavori su un programma preciso, controlla sempre la documentazione della versione che usi.

1 Concetti di base per orientarsi

Cinque idee bastano per collocare quasi ogni linguaggio: livello di astrazione, traduzione, tipi, memoria e paradigma.

Livello di astrazione

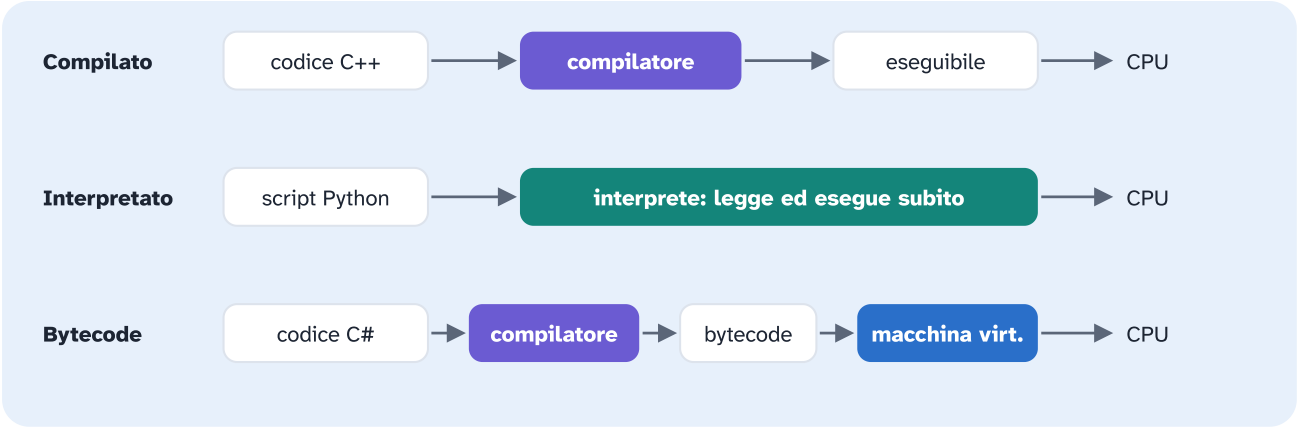
Più un linguaggio è «basso», più parla la lingua della macchina: registri, indirizzi di memoria, istruzioni del processore. Più è «alto», più parla la lingua del problema: muri, travi, liste di elementi, piani. Il basso livello è veloce ma faticoso; l'alto livello è comodo ma consuma più risorse. Ogni livello usa quello sotto senza doverne conoscere i dettagli: chi scrive Python non pensa ai transistor, come chi progetta un solaio non ricalcola la resistenza del singolo aggregato.



La scala dell'astrazione. Ogni gradino nasconde quello sotto. In alto si ragiona con muri e liste, in basso con tensioni e interruttori.

Compilato, interpretato o a metà strada

Un linguaggio **compilato** (C, C++, Rust, Go) viene tradotto una volta per tutte in un file eseguibile: è veloce, ma ogni modifica richiede una nuova compilazione. Uno **interpretato** (Python, JavaScript, i vecchi BASIC) viene letto ed eseguito al momento: più lento, ma immediato da provare. In mezzo c'è il **bytecode** su macchina virtuale (Java, C#): si compila per una macchina «astratta», che poi traduce per il processore reale. È per questo che un plug-in di Revit in C# va compilato, mentre uno script Python in Dynamo si modifica e si rilancia subito.



Tre modi di arrivare al processore. Il risultato finale è sempre lo stesso: istruzioni macchina. Cambia quando e da chi vengono tradotte.

Tipizzazione

Il **tipo** dice che cosa contiene una variabile: un numero intero, un decimale, un testo, una lista, un oggetto «Muro». Con la tipizzazione **statica** il tipo si dichiara e viene controllato prima di eseguire (C#, Java, TypeScript): più errori trovati subito, più parole da scrivere. Con quella **dinamica** il tipo si scopre durante l'esecuzione (Python, JavaScript): più rapida da scrivere, errori scoperti più tardi.

> **ANALOGIA DA CANTIERE**
>
> La tipizzazione statica è un abaco di Revit con i campi a valori controllati: non puoi scrivere «alto» dove serve un'altezza in metri. Quella dinamica è un foglio Excel libero: veloce da compilare, ma gli errori emergono quando qualcuno somma una cella che contiene testo.

Gestione della memoria

Ogni programma usa la memoria per tenere i suoi dati, e prima o poi deve liberarla. Nei linguaggi con **garbage collector** (Java, C#, Python, JavaScript) se ne occupa un meccanismo automatico. In C e C++ la gestisce chi scrive il codice: massimo controllo, ma è la fonte di molti errori e falle di sicurezza. Rust adotta una terza via: controlla l'uso della memoria già in compilazione, con regole di «proprietà» dei dati.

Paradigmi

I paradigmi sono i modi di ragionare con cui si scrive un programma. Quasi tutti i linguaggi moderni ne mescolano più d'uno.

PARADIGMA	IDEA DI FONDO	ANALOGIA DA CANTIERE	ESEMPI
Imperativo / procedurale	Una sequenza di ordini che modificano lo stato	Il cronoprogramma: fai questo, poi questo	C, Fortran, Pascal, Go
Orientato agli oggetti	Oggetti con dati (proprietà) e comportamenti (metodi), organizzati in classi ed ereditarietà	Le famiglie di Revit: categoria, famiglia, tipo, istanza, ognuna con i suoi parametri	Java, C#, C++, Python
Funzionale	Il programma è una catena di funzioni che producono nuovi dati senza modificare quelli esistenti	Una catena di verifiche dove ogni passaggio produce un nuovo risultato	Haskell, Lisp, F#, Elixir
Logico	Si dichiarano fatti e regole; il sistema deduce le risposte	Il controllo automatico di un modello rispetto alle regole di una norma	Prolog, Datalog
Dichiarativo	Si descrive *che cosa* si vuole, non *come* ottenerlo	Il capitolato prestazionale invece di quello descrittivo	SQL, HTML, espressioni regolari
Visuale	Si programma collegando blocchi invece di scrivere testo	Uno schema a blocchi di un impianto	Dynamo, Grasshopper, Scratch

2 Dal tasto al modello: elettricità, elettronica e codice

Selezioni un muro in Revit e premi `Canc`. Il muro sparisce. In mezzo, in una frazione di secondo, succedono almeno otto passaggi, dalla corrente elettrica al database del modello.



Dal dito al modello in otto passaggi. I primi quattro sono elettronica e protocolli, il quinto è il sistema operativo, gli ultimi tre sono il software che usi ogni giorno.

Passo per passo

1. **Il tasto chiude un circuito.** Sotto i tasti c'è una griglia di conduttori disposti in righe e colonne. Premendo, un contatto mette in comunicazione una riga e una colonna (nelle tastiere ottiche o capacitive cambia il principio, non il risultato): un piccolo passaggio di corrente diventa la prova che quel tasto è premuto.

2. **Il controllore della tastiera trova il tasto.** Un microcontrollore, cioè un minuscolo computer dentro la tastiera, alimenta le righe una alla volta e legge le colonne, moltissime volte al secondo. Quando trova un incrocio chiuso sa quale tasto è. Prima di crederci aspetta qualche millesimo di secondo, perché un contatto meccanico «rimbalza» e per un istante si apre e si chiude più volte: è il *debounce*. Poi traduce la posizione in un codice standard.

3. **Il messaggio viaggia sul cavo.** Con USB è il computer a interrogare la tastiera a intervalli regolari, di solito ogni pochi millesimi di secondo. La tastiera risponde con un piccolo pacchetto di dati, il *report*, definito dallo standard HID (*Human Interface Device*), che dice quali tasti sono premuti. Lo stesso standard vale per mouse, joystick e molte tastiere Bluetooth.

4 **L'hardware avvisa il processore.** Il controller USB del computer riceve i dati e manda alla CPU un *interrupt*, una specie di campanello: il processore sospende per un attimo ciò che stava facendo, esegue il driver della tastiera e poi riprende. Il driver è il primo pezzo di software della catena.

5 **Il sistema operativo crea un evento.** Windows traduce il codice del tasto in un tasto «virtuale» (indipendente dal modello di tastiera) e, tenendo conto del layout italiano, nel carattere corrispondente. Poi consegna l'evento alla finestra che ha il fuoco: per questo, se stai scrivendo in un'altra finestra, il muro non sparisce.

6 **L'applicazione decide che cosa significa.** Ogni programma con interfaccia grafica gira intorno a un *ciclo degli eventi*: aspetta un evento, lo gestisce, aspetta il successivo. Revit riceve «tasto Canc premuto» e lo collega al comando Elimina, come farebbe con una scorciatoia da tastiera che hai personalizzato.

7 **Il comando modifica il database del modello.** Un modello BIM è un database di oggetti collegati. Il comando apre una *transazione*: toglie il muro, aggiorna gli elementi che dipendono da lui (porte, finestre, quote, locali) e chiude la transazione. Se qualcosa va storto la transazione si annulla per intero: è il motivo per cui un comando non lascia mai il modello a metà, ed è lo stesso meccanismo su cui si basa il comando Annulla.

8 **Le viste si ridisegnano.** Il programma rigenera le viste interessate e passa la geometria alla scheda video (GPU), che calcola i pixel. Il monitor aggiorna l'immagine decine di volte al secondo e, al refresh successivo, il muro non c'è più.

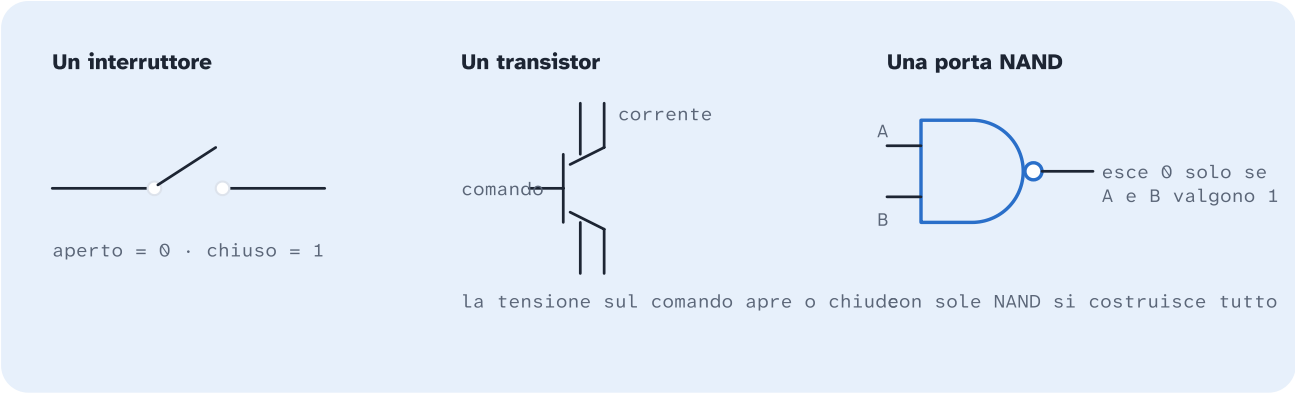
PERCHÉ INTERESSA A CHI FA BIM

Quando uno script Dynamo o un plug-in modifica il modello, salta i passi da 1 a 6 ma passa sempre dal 7: lavora dentro una transazione. Per questo l'API di Revit chiede di aprire e chiudere esplicitamente una transazione prima di cambiare qualcosa, e per questo un'operazione automatica si annulla tutta insieme con un solo «Annulla».

Che cosa vuol dire «elettronica che calcola»

Il transistor è un interruttore. Un transistor lascia passare corrente oppure no a seconda della tensione sul suo terminale di comando. Tensione alta vale 1, tensione bassa vale 0: è il **bit**. Un processore moderno contiene miliardi di transistor.

Le porte logiche fanno ragionamenti elementari. Pochi transistor collegati formano porte AND, OR e NOT. Con la sola porta NAND si può costruire qualsiasi altra funzione logica. È l'algebra di Boole (1854) diventata circuito: lo stesso `and`, `or`, `not` che leggerai in ogni `if`.



Dall'interruttore al ragionamento. Un bit è uno stato elettrico; una porta logica è un piccolo gruppo di interruttori che decide un'uscita in base agli ingressi.

Dalle porte al calcolo e alla memoria. Combinando porte si ottiene un sommatore, che somma due numeri binari. Collegandole «in retroazione», con l'uscita che torna all'ingresso, si ottiene un *flip-flop*, che ricorda un bit: è la base dei registri e della memoria.

Il processore. Contiene un'unità aritmetico-logica (ALU) che fa i calcoli, pochi registri velocissimi e un'unità di controllo che, a ogni colpo di clock, ripete il ciclo *preleva, decodifica, esegui*: legge un'istruzione dalla memoria, capisce che cosa chiede, la esegue, passa alla successiva. Un clock di 3 GHz scandisce tre miliardi di colpi al secondo.

L'architettura di von Neumann (1945). Programma e dati stanno nella stessa memoria, organizzata come una fila di celle numerate (gli indirizzi). Quasi tutti i computer seguono ancora questo schema, ed è il motivo per cui la programmazione imperativa, cioè una sequenza di ordini che modificano celle di memoria, è la più diffusa: rispecchia la macchina.

Il set di istruzioni. Ogni famiglia di processori ha il suo vocabolario di istruzioni elementari: x86-64 per la maggior parte dei PC con Windows, ARM per telefoni, tablet e Mac recenti. Il **codice macchina** sono queste istruzioni in binario; l'**assembly** è la stessa cosa scritta con sigle leggibili, una riga per istruzione.

Che cosa si porta dietro ogni linguaggio dall'hardware

COSA VEDI NEL CODICE	COSA SUCCEDE SOTTO
Variabile `x = 5`	Una cella di memoria a un certo indirizzo, in cui si scrive 5 in binario
Tipi `int`, `float`, `bool`, `string`	Istruzioni diverse del processore: la stessa sequenza di bit può essere un intero o un decimale, a seconda del tipo
Intero che «impazzisce» oltre un limite	Un intero a 32 bit ha posto per circa ±2,1 miliardi; oltre, i bit ricominciano (*overflow*)
`0.1 + 0.2` **che non fa** `0.3`	I decimali sono in virgola mobile binaria (norma IEEE 754): lo vediamo nel capitolo 3
Liste che partono da `0`	L'indice è lo spostamento dall'indirizzo iniziale: il primo elemento è a spostamento `0`
`if` **/** `else`	Un salto condizionato: «se il registro vale `0`, salta all'istruzione n»
Cicli `for` **/** `while`	Un salto all'indietro ripetuto finché la condizione è vera
Funzioni	Lo *stack*: si salva il punto di ritorno, si salta alla funzione, poi si torna. Una ricorsione senza fine esaurisce lo stack (*stack overflow*)
`null` **/** `None`	Un riferimento che non punta a niente: usarlo genera l'errore più comune di tutti
Thread, `async`**, parallelismo**	Da metà anni `2000` la frequenza dei processori cresce poco: si aumentano i core. Molti linguaggi recenti nascono anche per sfruttarli in sicurezza
Calcolo su GPU	Migliaia di piccoli core che fanno la stessa operazione su molti dati: grafica, rendering, intelligenza artificiale

3 Bit, numeri e coordinate

Tutto, nel computer, è una fila di 0 e 1: lettere, colori, coordinate, modelli. Capire come vengono rappresentati i numeri evita errori molto concreti nei modelli e nei fogli di calcolo.



Testo e numeri sono codici. Una lettera è un numero in una tabella condivisa (ASCII, oggi Unicode); un decimale è un segno, un esponente e un numero limitato di cifre.

Il testo: da ASCII a Unicode

Per scrivere una lettera il computer usa un numero preso da una tabella condivisa. La tabella storica, ASCII, aveva 128 caratteri: niente lettere accentate. Oggi si usa Unicode, che assegna un numero a oltre centomila caratteri di tutte le lingue, e quasi sempre la codifica UTF-8 per salvarli. Quando in un file esportato compaiono simboli strani al posto delle accentate (`Ã¨` invece di `è`), un programma ha letto in un modo un testo scritto in un altro.

I decimali: perché 0,1 + 0,2 non fa 0,3

I numeri decimali sono salvati in **virgola mobile binaria**, secondo la norma IEEE 754. Come 1/3 in base dieci diventa 0,3333… senza fine, così 0,1 in base due è un numero periodico: il computer lo tronca, e il risultato di `0.1 + 0.2` è `0.30000000000000004`. Non è un difetto di un linguaggio: succede in Python, in C#, in JavaScript e anche in Excel.

REGOLA PRATICA

Non confrontare mai due decimali con `==`. Confrontali con una tolleranza: `abs(a - b) < 1e-9`. Vale per le lunghezze lette da un modello, per le aree e per i risultati di calcolo.

Coordinate lontane dall'origine

Un decimale a 64 bit ha circa 15–16 cifre significative in tutto. Se una coordinata è molto grande, per esempio una coordinata geografica espressa in millimetri, le cifre «si consumano» prima della virgola e ne restano meno per i decimali. Molti programmi di grafica lavorano poi internamente con numeri a 32 bit, che di cifre ne hanno circa sette. Ecco perché i software BIM consigliano di modellare vicino all'origine interna e di gestire la posizione reale con un sistema di coordinate condiviso, invece di spostare la geometria a chilometri di distanza.

UNITÀ INTERNE

I programmi non salvano «metri» o «millimetri»: salvano numeri in un'unità interna e li convertono quando li mostrano. L'API di Revit, per esempio, restituisce le lunghezze in piedi e gli angoli in radianti, qualunque siano le unità del progetto: uno script che legge una lunghezza e la scrive in un foglio senza convertirla sbaglia di un fattore 0,3048. Controlla sempre l'unità interna del programma su cui lavori.

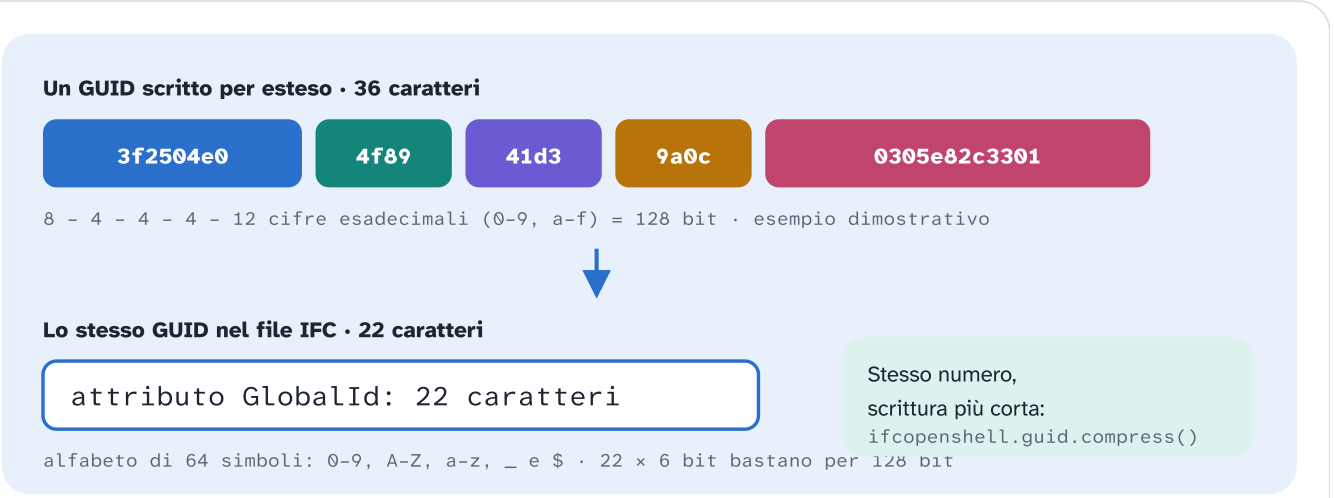
4 Il GUID: l'identità degli oggetti

Un muro può cambiare nome, tipo, spessore, livello. Quello che non deve cambiare mai è la sua identità. Il GUID serve a questo.

Che cos'è

GUID significa *Globally Unique Identifier*, identificatore unico globale; lo stesso concetto si chiama anche UUID (*Universally Unique Identifier*). È un numero a **128 bit**, di solito scritto come 32 cifre esadecimali divise in cinque gruppi da 8, 4, 4, 4 e 12 caratteri. Lo standard di riferimento è l'RFC 9562, pubblicato nel maggio 2024, che ha sostituito il precedente RFC 4122.

La forza del GUID è che **non serve un registro centrale**: ogni programma, su ogni computer, può generarne uno da solo con la certezza pratica che nessun altro, al mondo, ne generi uno uguale. Nella versione più comune (la 4) quasi tutti i bit sono casuali: le combinazioni possibili sono così tante che, per avere anche solo una probabilità su due di trovare due GUID uguali, bisognerebbe generarne miliardi di miliardi.

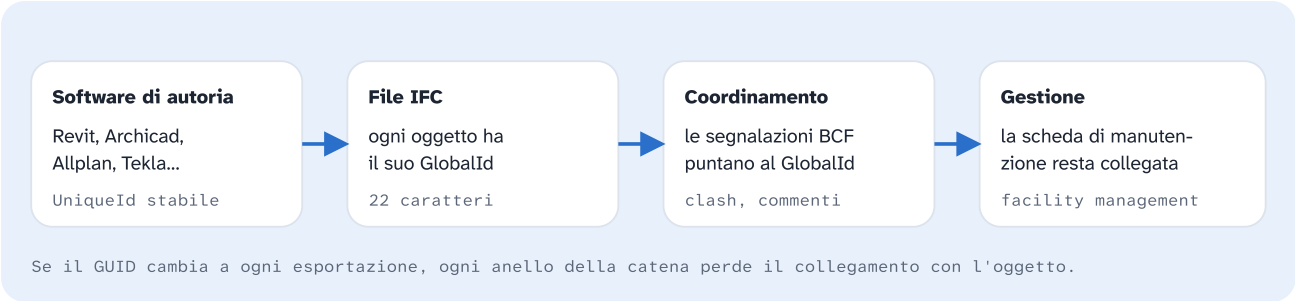


Due modi di scrivere lo stesso numero. Nel formato IFC il GUID viene compresso in 22 caratteri con un alfabeto di 64 simboli: per questo i GlobalId sembrano stringhe casuali.

Il GUID nel BIM

Nel formato **IFC** ogni oggetto che deriva da `IfcRoot` (muri, solai, porte, spazi, ma anche relazioni e proprietà) ha un attributo obbligatorio `GlobalId`: un GUID di 128 bit compresso in 22 caratteri. È questo valore, non il nome né il tipo, a dire «questo è lo stesso muro» tra due versioni del modello.

In **Revit** ogni elemento ha due identificativi. L'`ElementId` è un numero valido solo dentro quel file. L'`UniqueId` è una stringa più lunga, formata da un GUID e dall'identificativo dell'elemento in esadecimale, ed è stabile nel tempo: resta lo stesso anche dopo aggiornamenti di versione e sincronizzazioni con il modello centrale. In esportazione IFC il programma ricava da qui il GlobalId, e l'esportatore può anche salvarlo in un parametro dell'elemento.



Il filo che tiene insieme il processo. Il GUID permette di riconoscere lo stesso oggetto in programmi, versioni e fasi diverse.

Perché è così importante

- **Confrontare due versioni del modello.** Gli strumenti di confronto riconoscono che un oggetto è stato modificato, e non cancellato e ricreato, proprio grazie al GlobalId.
- **Coordinamento e BCF.** Una segnalazione di interferenza in formato BCF dice a quali oggetti si riferisce indicandone il GlobalId: se l'oggetto cambia identità alla versione successiva, la segnalazione resta orfana.
- **Collegare dati esterni.** Computi, programmi di manutenzione, database di gestione: tutto ciò che si aggancia al modello dall'esterno usa il GUID come chiave, come un codice fiscale dell'oggetto.
- **Tracciabilità nel tempo.** Dal progetto al cantiere alla gestione, l'oggetto cambia livello di dettaglio, ma resta lo stesso oggetto.

ERRORI DA EVITARE

- Ricreare un elemento invece di modificarlo: il nuovo oggetto avrà un nuovo GUID.
- Usare l'ElementId di Revit come chiave tra file diversi: vale solo dentro il suo file.
- Copiare oggetti in un altro modello e aspettarsi che mantengano l'identità.
- Modificare a mano i GlobalId in un file IFC: due oggetti con lo stesso GUID rendono il file non valido.
- Fidarsi senza controllare: dopo due esportazioni dello stesso modello, verifica su qualche oggetto che il GlobalId sia rimasto uguale.

Il GUID nel codice

Python · creare un UUID e convertirlo nel formato IFC con IfcOpenShell

```
import uuid
import ifcopenshell.guid

u = uuid.uuid4()                                    # un nuovo UUID casuale
(versione 4)
print(u)                                            # 36 caratteri con i
trattini
print(ifcopenshell.guid.compress(u.hex))            # gli stessi 128 bit in 22
caratteri IFC
print(ifcopenshell.guid.new())                      # un GlobalId IFC nuovo,
in un solo passaggio
```

Python · leggere i GlobalId dei muri di un file IFC

```
import ifcopenshell

modello = ifcopenshell.open("scuola.ifc")
for muro in modello.by_type("IfcWall"):
    print(muro.GlobalId, muro.Name)
```

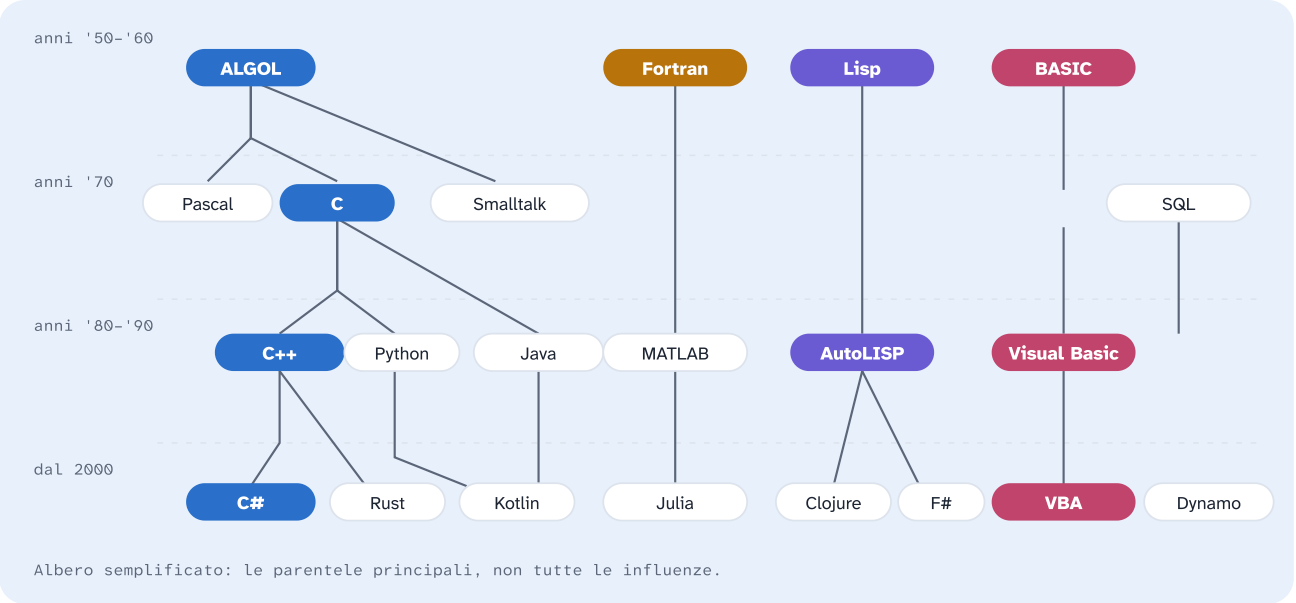
C# · leggere l'UniqueId degli elementi in un plug-in di Revit

```
var muri = new FilteredElementCollector(doc)
    .OfCategory(BuiltInCategory.OST_Walls)
    .WhereElementIsNotElementType();

foreach (Element m in muri)
{
    string id = m.UniqueId;   // stabile nel tempo, a differenza di m.Id
}
```

5 La mappa dei linguaggi

Esistono migliaia di linguaggi, ma quasi tutti discendono da pochi antenati. Riconoscere la parentela è il modo più rapido per leggerli.



Quattro antenati, molte famiglie. Dalla linea di ALGOL e del C viene la sintassi con graffe e punto e virgola; da Lisp viene AutoLISP; da BASIC viene il VBA dei fogli Excel; da Fortran il calcolo scientifico.

A che cosa servono, per ambito

La domanda utile spesso non è «quando è nato» ma «a che cosa serve». Lo stesso linguaggio compare in più ambiti: Python è quasi ovunque, ed è il motivo per cui è il primo da imparare a leggere.

Automazione BIM

Python (Dynamo, pyRevit), C# (plug-in Revit, Navisworks, Tekla), linguaggi visuali

CAD

AutoLISP, script di AutoCAD, .NET in C#, GDL per Archicad

Dati del modello

Python con IfcOpenShell, SQL, JSON, espressioni regolari

Fogli di calcolo

Formule e LAMBDA di Excel, VBA, Power Query, Python in Excel

Calcolo tecnico

Python con NumPy e SciPy, MATLAB, Julia, Fortran nei solutori storici

Web e visualizzatori

JavaScript e TypeScript, HTML e CSS, visualizzatori IFC nel browser

Impianti e building automation

Linguaggi IEC 61131-3 per PLC, C sui microcontrollori

Intelligenza artificiale

Python con le sue librerie; C++ e CUDA sotto, per la velocità

Sistemi e prestazioni

C, C++, Rust, Go: il motore dei programmi che usi

6 Basso livello e imperativi

I linguaggi più vicini al processore e il grande ceppo da cui discende quasi tutto il resto.

Basso livello

Nessuno li sceglie per comodità: si usano dove servono controllo totale o massima velocità.

LINGUAGGIO	ANNO	DOVE SI USA	PUNTI DI FORZA	PUNTI DEBOLI
Codice macchina	anni '40	Ciò che il processore esegue davvero; nessuno lo scrive a mano	È l'unica cosa che la CPU capisce	Illeggibile: solo sequenze di bit
Assembly	~1949	Parti critiche di sistemi operativi, firmware, microcontrollori, analisi di virus	Controllo completo e velocità massima	Lunghissimo da scrivere, diverso per ogni famiglia di processori

```
mov eax, 5      ; metti 5 nel registro eax
add eax, 3      ; aggiungi 3: eax vale 8
```

Imperativi e procedurali

Il programma è una sequenza di istruzioni raggruppate in procedure. La sintassi del C (graffe, punto e virgola) la ritrovi in C++, Java, C#, JavaScript, PHP, Go e Rust.

LINGUAGGIO	ANNO	DOVE SI USA	PUNTI DI FORZA	PUNTI DEBOLI
Fortran	1957	Calcolo scientifico e ingegneristico: solutori agli elementi finiti, meteorologia, fluidodinamica; molti codici di calcolo strutturale storici	Velocissimo sui calcoli numerici, librerie collaudate da decenni	Sintassi datata, poco adatto a interfacce e testo
COBOL	1959	Banche, assicurazioni, sistemi gestionali storici della pubblica amministrazione	Stabile, preciso sui calcoli contabili	Prolisso; pochi programmatori giovani lo conoscono
ALGOL	1958–60	Oggi nessun uso: è il «nonno» di Pascal, C e di quasi tutti i linguaggi moderni	Ha introdotto i blocchi annidati e una grammatica formale	Mai diffuso nell'industria
BASIC	1964	Didattica e primi personal computer; vive in Visual Basic e VBA	Facilissimo da imparare	Le versioni storiche incoraggiavano codice disordinato
Pascal / Delphi	1970	Didattica; Delphi per software gestionale desktop	Rigoroso e leggibile	Oggi di nicchia
C	1972	Sistemi operativi, firmware, microcontrollori, motori di altri linguaggi (l'interprete standard di Python è scritto in C)	Veloce, portabile, «lingua franca» dei sistemi	Memoria gestita a mano: errori pericolosi
Ada	1980	Aeronautica, ferrovie, difesa, sistemi critici	Controlli rigorosissimi	Verboso, comunità piccola
Go	2009	Server e servizi cloud (Docker e Kubernetes sono scritti in Go)	Semplice, compila in fretta, buono per il lavoro in parallelo	Volutamente minimale
Zig	2016	Programmazione di sistema, alternativa moderna al C	Semplice, con più controlli del C	Giovane e ancora in evoluzione

7 Orientati agli oggetti

La famiglia dominante nel software commerciale, comprese le API di Revit, Archicad, Allplan e Tekla.

Il programma è un insieme di **oggetti** che hanno proprietà e metodi e appartengono a **classi**; le classi possono ereditare da altre.

SE CONOSCI REVIT CONOSCI GIÀ L'IDEA

La classe `Wall` ha proprietà (spessore, altezza, livello) e metodi (unisci, dividi). Ogni muro del progetto è un'**istanza** di quella classe. Un muro strutturale **eredita** tutto dal muro generico e aggiunge le sue proprietà: è la stessa logica di categoria, famiglia, tipo e istanza.

LINGUAGGIO	ANNO	DOVE SI USA	PUNTI DI FORZA	PUNTI DEBOLI
Simula	1967	Nato per le simulazioni, ha inventato classi e oggetti	Ha introdotto il paradigma	Solo interesse storico
Smalltalk	1972	Didattica e ricerca; ha ispirato le interfacce grafiche moderne	«Tutto è un oggetto», molto coerente	Uso industriale marginale
C++	1985	Software tecnico pesante: CAD, motori di calcolo e di rendering, videogiochi; il nucleo di molti programmi BIM	Velocità del C con le classi	Molto complesso, memoria a mano
Objective-C	1984	App Apple storiche, sostituito da Swift	Base di macOS e iOS per decenni	In declino
Java	1995	Gestionali aziendali, banche, Android, server; molti software della pubblica amministrazione	«Scrivi una volta, gira ovunque», ecosistema enorme	Verboso, consumo di memoria
C#	2000	Ecosistema .NET, applicazioni Windows, videogiochi (Unity), **plug-in per Revit, Navisworks, AutoCAD, Tekla**	Moderno, ottimi strumenti di sviluppo	Nato legato a Microsoft, oggi multipiattaforma
VB.NET	2002	Plug-in .NET e gestionali	Sintassi vicina al vecchio Visual Basic	In declino rispetto a C#
Kotlin	2011	App Android (il linguaggio preferito da Google), server	Più conciso e sicuro di Java	Legato all'ecosistema Java
Dart	2011	App multipiattaforma con Flutter	Una sola base di codice per Android, iOS e web	Usato quasi solo con Flutter
Swift	2014	App per iPhone, iPad, Mac	Sicuro, moderno, veloce	Quasi solo nel mondo Apple
Rust	2015 (1.0)	Sistemi, browser, strumenti da riga di comando, parti di	Velocità del C++ con la sicurezza	Difficile da imparare

	Windows e Linux	della memoria garantita

Anche Python, JavaScript, PHP e Ruby sono orientati agli oggetti, ma li trovi tra i linguaggi di scripting, per il loro uso tipico.

8 Funzionali, logici e dichiarativi

Funzionali

Il programma è una composizione di funzioni che prendono dati e restituiscono dati nuovi, senza modificare quelli esistenti. Si usano meno direttamente, ma le loro idee sono ovunque: `map`, `filter`, le funzioni *lambda* e le liste immutabili di Python, C#, JavaScript e delle formule LAMBDA di Excel vengono da qui. Anche Dynamo ragiona così: ogni nodo riceve dati e ne produce di nuovi.

LINGUAGGIO	ANNO	DOVE SI USA	PUNTI DI FORZA	PUNTI DEBOLI
Lisp	1958	Intelligenza artificiale storica; vive in AutoLISP ed Emacs	Il codice è una lista di dati: estremamente flessibile	Tante parentesi
Scheme / Racket	1975	Didattica universitaria	Minimale ed elegante	Poco uso industriale
ML / OCaml	1973 / 1996	Compilatori, verifica formale, finanza	Tipi controllati senza doverli scrivere	Comunità piccola
Erlang	1986	Telecomunicazioni, messaggistica	Sistemi che non si fermano mai	Sintassi ostica
Haskell	1990	Ricerca, finanza, strumenti dove la correttezza è critica	Puro e rigoroso	Concetti astratti, curva ripida
Scala	2004	Elaborazione di grandi volumi di dati	Oggetti e funzionale sulla piattaforma Java	Complesso
F#	2005	Finanza, analisi dati in .NET	Funzionale ma integrato con tutto .NET	Poco diffuso rispetto a C#
Clojure	2007	Servizi web e dati su piattaforma Java	Lisp moderno	Nicchia
Elixir	2012	Applicazioni web in tempo reale	Robustezza di Erlang con sintassi moderna	Ecosistema più piccolo

Logici e dichiarativi

Si descrive *che cosa* si vuole e il sistema decide *come* ottenerlo. SQL è di gran lunga il più importante: ogni gestionale, ogni sito, ogni database lo usa, anche quelli che archiviano i dati di un modello.

LINGUAGGIO	ANNO	DOVE SI USA	PUNTI DI FORZA	PUNTI DEBOLI
SQL	1974	Interrogare e modificare database: gestionali, siti, archivi tecnici	Standard universale, poche parole per domande complesse	Ogni database ha il suo dialetto
Prolog	1972	Intelligenza artificiale simbolica, sistemi esperti	Basta dichiarare fatti e regole	Difficile da controllare su problemi grandi
Datalog	anni '70–'80	Interrogazioni su grafi di dati, verifica di regole	Più prevedibile di Prolog	Nicchia specialistica

```
-- quanti muri ci sono per piano, in un database di elementi
SELECT Livello, COUNT(*) AS Muri
FROM Elementi
WHERE Categoria = 'Muri'
GROUP BY Livello;
```

9 Scripting e web

Linguaggi interpretati, rapidi da scrivere, nati per automatizzare e collegare programmi. Sono quelli che incontrerai più spesso.

LINGUAGGIO	ANNO	DOVE SI USA	PUNTI DI FORZA	PUNTI DEBOLI
Bash (shell)	1989	Automazione su Linux e macOS, server	Combina programmi esistenti in poche righe	Sintassi criptica
PowerShell	2006	Automazione di Windows e Microsoft 365	Lavora con oggetti, non solo testo	Quasi solo mondo Microsoft
Perl	1987	Elaborazione di testo, vecchi script	Potentissimo con il testo	Difficile da rileggere; in declino
Python	1991	Ovunque: analisi dati, intelligenza artificiale, automazione, calcolo; nel BIM in Dynamo, pyRevit, Rhino e Grasshopper, Blender, Allplan, QGIS, IfcOpenShell	Leggibile quasi come pseudocodice, librerie per tutto	Lento sul calcolo puro (si compensa con librerie scritte in C); l'indentazione fa parte della sintassi
Ruby	1995	Siti web (Ruby on Rails); in passato i plug-in di SketchUp	Elegante e produttivo	In calo di popolarità
PHP	1995	Siti web lato server, WordPress	Disponibile ovunque	Fama di codice disordinato, molto migliorato
JavaScript	1995	L'unico linguaggio eseguito nativamente dai browser; anche server (Node.js) e visualizzatori BIM web	Gira ovunque ci sia un browser	Molte stranezze storiche (`"2" + 2` fa `"22"`)
TypeScript	2012	Applicazioni web medio-grandi	JavaScript con i tipi: errori trovati prima	Richiede una fase di traduzione
Lua	1993	Linguaggio incorporato in altri programmi: videogiochi, plug-in, configurazioni	Piccolissimo e veloce da integrare	Libreria standard minima

10 Calcolo scientifico e analisi dati

Linguaggi pensati per chi fa conti: vettori e matrici sono di casa, i grafici dei risultati sono immediati.

LINGUAGGIO	ANNO	DOVE SI USA	PUNTI DI FORZA	PUNTI DEBOLI
MATLAB	1984	Ingegneria, controllo, elaborazione dei segnali, università	Matrici native, toolbox specialistici	Licenza costosa
GNU Octave	1993	Alternativa libera a MATLAB	Gratuito, sintassi quasi identica	Meno strumenti
R	1993	Statistica e ricerca	Il riferimento per l'analisi statistica	Sintassi irregolare
Mathematica	1988	Calcolo simbolico, matematica, fisica	Risolve equazioni in forma simbolica	Commerciale, sistema chiuso
Julia	2012	Calcolo ad alte prestazioni, simulazioni	Facile come Python, veloce quasi come C	Ecosistema giovane

Nella pratica di oggi molto di questo lavoro si fa in Python con NumPy, SciPy, pandas e Matplotlib: per un ufficio tecnico è spesso la scelta più economica e più condivisa.

11 I linguaggi del BIM e dell'ufficio tecnico

Quasi nessuno è un linguaggio «nuovo»: sono linguaggi generali applicati alle API dei software tecnici, più alcuni ambienti visuali.

LINGUAGGIO O AMBIENTE	BASE	DOVE SI USA	PUNTI DI FORZA	PUNTI DEBOLI
C# per le API	C# / .NET	Plug-in di Revit, Navisworks, AutoCAD, Tekla	Accesso completo all'API, interfacce vere	Serve un ambiente di sviluppo e la compilazione
Python in Revit	Python	pyRevit e simili: rinomina, controlli, esportazioni	Veloce da scrivere e modificare	Meno strumenti rispetto a C#
Dynamo	Visuale + nodi Python	Automazioni in Revit e Civil 3D senza partire da zero	Accessibile e visivo	Grafi grandi diventano illeggibili e lenti; dipendenza dai pacchetti
Grasshopper	Visuale + C# / Python	Progettazione parametrica in Rhino, forme complesse, ottimizzazione	Potentissimo per la geometria	Meno orientato ai dati BIM
AutoLISP	Lisp	Comandi personalizzati di AutoCAD	Moltissime routine già scritte	Sintassi datata
Script di AutoCAD (.scr)	Sequenza di comandi	Ripetere operazioni su molti disegni	Nessuna programmazione: si registrano i comandi	Nessuna logica (niente condizioni né cicli)
GDL	Linguaggio proprio	Oggetti parametrici di Archicad	Controllo completo dell'oggetto 2D e 3D	Solo Archicad
PythonParts	Python	Oggetti parametrici e automazioni in Allplan	Usa Python standard	Comunità più piccola
IfcOpenShell	Python / C++	Leggere, controllare e modificare file IFC con qualsiasi software	Libero, indipendente dal produttore	Serve conoscere lo schema IFC

VBA	Visual Basic	Macro di Excel, Word, Access; fogli di calcolo tecnici	Già dentro Office	Linguaggio vecchio, rischi di sicurezza delle macro
Formule, LAMBDA, Power Query	Dichiarativo e funzionale	Calcoli e trasformazioni di dati in Excel	Nessuna installazione	Difficili da verificare quando crescono
EXPRESS	ISO 10303-11	Definisce lo schema IFC: entità e attributi	Formale e rigoroso	Descrive dati, non esegue nulla
IEC 61131-3	Standard per PLC	Building automation, controlli degli impianti	Standard internazionale per tecnici di impianti	Ogni costruttore ha i suoi ambienti

Lo stesso compito in quattro linguaggi

Contare i muri del modello, quattro volte. Guarda come cambiano le parole e come resta uguale l'idea.

Python con IfcOpenShell · su un file IFC

```
import ifcopenshell
m =
ifcopenshell.open("modello.ifc")
muri = m.by_type("IfcWall")
print(len(muri), "muri")
```

C# · dentro un plug-in di Revit

```
int n = new
FilteredElementCollector(doc)

.OfCategory(BuiltInCategory.OST_Walls
    .WhereElementIsNotElementType()
    .GetElementCount();
```

Python in un nodo di Dynamo

```
# IN[0]: la lista dei muri da un
nodo precedente
muri = IN[0]
OUT = len(muri)
```

AutoLISP · contare le polilinee di un disegno

```
(defun c:CONTA ( / s)
  (setq s (ssget "_X" '((0 .
"LWPOLYLINE"))))
  (princ (if s (sslength s) 0))
  (princ))
```

LE API CAMBIANO

Le API di Revit e di Dynamo cambiano tra le versioni annuali, e così i pacchetti della comunità. Uno script che funziona su una versione può richiedere modifiche sulla successiva: tieni traccia della versione per cui l'hai scritto.

12 Non sono linguaggi di programmazione, ma li leggerai

Descrivono dati, aspetto o struttura, ma non contengono logica: niente condizioni né cicli.

FORMATO	CHE COSA DESCRIVE	DOVE LO TROVI	DA SAPERE
IFC-SPF (.ifc)	Il modello BIM in testo, una riga per entità	File IFC aperti in un editor di testo	Ogni riga è del tipo `#123=IFCWALL(...)`, con riferimenti ad altre righe
XML	Dati strutturati ad albero	ifcXML, gbXML per l'energetica, BCF, configurazioni	Prolisso ma rigoroso
JSON	Coppie chiave–valore e liste	API web, configurazioni, file di Dynamo (.dyn)	Più leggero di XML, è lo standard del web
CSV	Tabelle in testo semplice	Esportazioni da Revit, Excel, database	Il separatore cambia tra paesi: attenzione ai decimali con la virgola
HTML e CSS	Struttura e aspetto di una pagina web	Ogni sito	Marcatori tra parentesi angolari e regole di stile
YAML	Come JSON, ma più leggibile	Configurazioni di software e server	L'indentazione ha significato
Markdown	Testo formattato in modo semplice	Documentazione, note	`# titolo`, `**grassetto**`
Espressioni regolari	Modelli di ricerca nel testo	Cerca e sostituisci avanzato, rinomina in Revit e Dynamo	Potentissime ma poco leggibili: `^\d{3}-[A-Z]+$`

13 Come leggere codice che non conosci

Quasi tutti i linguaggi usano gli stessi sette mattoni con parole diverse. Se li riconosci, segui il filo di un programma anche senza conoscerne il linguaggio.

MATTONE	PYTHON	C#	COSA FA
Variabile	`h = 3.0`	`double h = 3.0;`	Dà un nome a un valore
Condizione	`if h > 2.7:`	`if (h > 2.7) { }`	Esegue un blocco solo se vale una condizione
Ciclo	`for muro in muri:`	`foreach (var muro in muri) { }`	Ripete per ogni elemento
Funzione	`def area(b, h): return b*h`	`double Area(double b, double h) => b*h;`	Un pezzo di codice riutilizzabile
Lista	`muri = [m1, m2]`	`List<Wall> muri`	Più valori in un contenitore
Oggetto e metodo	`muro.Width`	`muro.Width`	Proprietà e azioni di un oggetto
Gestione degli errori	`try: … except:`	`try { } catch { }`	Che cosa fare se qualcosa va storto

Metodo di lettura in cinque passi

1. **Trova il punto di ingresso.** In Python spesso è il fondo del file; in un plug-in di Revit il metodo `Execute`; in Dynamo il primo nodo a sinistra.

2. **Leggi i nomi prima del contenuto.** Un buon codice si racconta dai nomi delle funzioni e delle variabili.

3. **Segui i dati.** Da dove entrano (file, modello, utente), come si trasformano, dove escono.

4. **All'inizio ignora i dettagli tecnici.** Import, configurazioni e gestione degli errori si guardano dopo.

5 **Leggi i commenti, ma verificali.** `#` in Python, `//` in C#: non sempre sono aggiornati quanto il codice.

La stessa idea in sette linguaggi

CONCETTO	PYTHON	JAVASCRIPT	C# / JAVA	C	VBA	SQL
Commento	`# nota`	`// nota`	`// nota`	`/* nota */`	`' nota`	`-- nota`
Variabile	`x = 5`	`let x = 5`	`int x = 5;`	`int x = 5;`	`Dim x As Integer`	una colonna
Uguaglianza	`==`	`===`	`==`	`==`	`=`	`=`
Blocco	indentazione	`{ }`	`{ }`	`{ }`	`If … End If`	—
Condizione	`if x > 0:`	`if (x > 0) {}`	`if (x > 0) {}`	`if (x > 0) {}`	`If x > 0 Then`	`WHERE x > 0`
Ciclo	`for m in muri:`	`for (const m of muri)`	`foreach (var m in muri)`	`for (i = 0; i < n; i++)`	`For Each m In muri`	implicito, su tutte le righe
Funzione	`def f(a):`	`function f(a) {}`	`int F(int a) {}`	`int f(int a) {}`	`Function F(a)`	`CREATE FUNCTION`
Valore nullo	`None`	`null`, `undefined`	`null`	`NULL`	`Nothing`	`NULL`

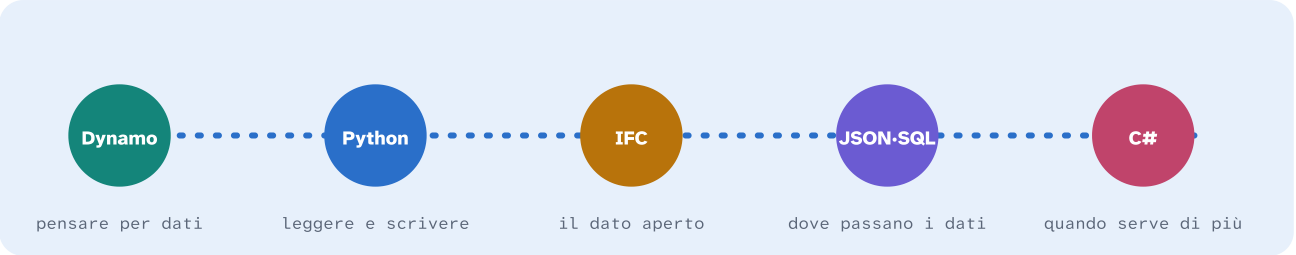
Chi assomiglia a chi

- **C → C++ → Java → C#**: quattro generazioni della stessa famiglia.
- **Java e C#** sono quasi gemelli; **JavaScript e Java**, invece, non sono parenti: il nome fu una scelta commerciale del 1995.
- **TypeScript** sta a JavaScript come un abaco con campi controllati sta a un foglio libero.

- **Kotlin** sta a Java come **Swift** sta a Objective-C: la versione moderna dello stesso ecosistema.
- **Python e Ruby** sono cugini; **Python e MATLAB** si sovrappongono molto nel calcolo tecnico.
- **Go e Rust** sono i due eredi moderni del C: uno punta alla semplicità, l'altro alla sicurezza.
- **Dynamo e Grasshopper** sono la stessa idea, programmazione a nodi, applicata a Revit e a Rhino.
- **SQL e i filtri di Revit o di Excel** fanno la stessa cosa: scelgono le righe che rispettano una condizione.
- **Lisp e AutoLISP**: il secondo è un dialetto del primo, adattato ad AutoCAD.

14 Da dove cominciare

Un percorso pensato per chi lavora in uno studio o in un'impresa e vuole automatizzare, controllare e capire i dati dei modelli.



Cinque tappe. Non è obbligatorio arrivare in fondo: già le prime due bastano per automatizzare buona parte del lavoro ripetitivo.

1. **Dynamo.** Abitua a ragionare per liste di elementi e trasformazioni, senza scrivere codice.

2. **Python.** È il più leggibile e il più usato nel BIM: entra in Dynamo, in pyRevit, in Rhino, in Blender e nella lettura degli IFC.

3. **IFC con IfcOpenShell.** Insegna a lavorare sul dato aperto, indipendente dal software di partenza, e a capire davvero il GUID.

4. **JSON e SQL.** I dati passano quasi sempre da lì: configurazioni, API web, database dei computi e della gestione.

5. **C#.** Quando serve un plug-in vero, con interfaccia e prestazioni, la documentazione dell'API di Revit è scritta in C#.

UN CONSIGLIO DI METODO

Parti da un problema reale e ripetitivo del tuo lavoro: rinominare tavole, controllare parametri vuoti, esportare quantità. Un piccolo script che fa risparmiare mezz'ora alla settimana insegna più di un corso generico, e si migliora un pezzo alla volta.

15 Glossario essenziale

TERMINE	SIGNIFICATO IN BREVE
Algoritmo	Sequenza finita di passi per risolvere un problema; esiste prima e indipendentemente dal linguaggio
API	Insieme di funzioni che un programma mette a disposizione di altri (l'API di Revit permette ai plug-in di leggere e modificare il modello)
Bit, byte	Il bit è un singolo 0 o 1; un byte sono 8 bit
Booleano	Valore vero o falso
Bug, debug	Errore nel codice; ricerca e correzione dell'errore
Bytecode	Codice intermedio eseguito da una macchina virtuale
Classe, istanza	Lo «stampo» e il singolo oggetto creato da esso, come famiglia e istanza in Revit
Compilatore	Programma che traduce tutto il codice in eseguibile prima di avviarlo
Dizionario	Collezione di coppie chiave → valore
Driver	Programma che fa dialogare il sistema operativo con un dispositivo
Ereditarietà	Una classe riceve proprietà e metodi da un'altra
Eccezione	Errore che interrompe il flusso e si può intercettare
Firmware	Software scritto dentro un dispositivo, come la tastiera o una caldaia
Framework	Struttura già pronta su cui si costruisce un'applicazione
Funzione, metodo	Blocco di codice riutilizzabile; è un metodo quando appartiene a un oggetto
Garbage collector	Meccanismo che libera da solo la memoria non più usata
Git	Sistema di controllo delle versioni: chi ha cambiato che cosa, e quando
GUID, UUID	Identificatore a 128 bit unico nel mondo, senza bisogno di un registro centrale
IDE	Ambiente di sviluppo: editor, compilatore, debugger insieme

Interprete	Programma che esegue il codice senza produrre un eseguibile
Interrupt	Segnale con cui un dispositivo chiede l'attenzione del processore
Libreria, pacchetto	Codice scritto da altri, pronto da riutilizzare
Namespace, modulo	Contenitore che raggruppa nomi ed evita conflitti
Overflow	Valore che supera la capacità del tipo che lo contiene
Ricorsione	Funzione che chiama se stessa
Runtime	Il momento, e l'ambiente, in cui il programma è in esecuzione
Script	Programma breve, di solito interpretato, per automatizzare un compito
Stack, heap	Le due zone di memoria: ordinata per le chiamate di funzione, libera per i dati
Stringa	Sequenza di caratteri
Thread	Filo di esecuzione; più thread lavorano in parallelo
Transazione	Gruppo di modifiche che riesce per intero o si annulla per intero
Virgola mobile	Il modo in cui il computer salva i decimali: segno, esponente e cifre significative

16 Curiosità che insegnano qualcosa

Dietro quasi ogni aneddoto della storia del software c'è un limite del calcolatore o un'ipotesi non verificata: lezioni valide anche per chi progetta.

Errori celebri

Mars Climate Orbiter, 1999

La sonda della NASA andò perduta perché un software forniva gli impulsi in unità anglosassoni e un altro li leggeva in unità SI. Nessun linguaggio controllava le unità di misura: vale per ogni foglio di calcolo strutturale.

Ariane 5, volo 501, 1996

Il razzo si autodistrusse dopo circa 40 secondi: un valore decimale a 64 bit fu convertito in un intero a 16 bit troppo piccolo. Il codice era stato riusato dall'Ariane 4 senza riverificarne le ipotesi.

Missili Patriot, 1991

Il sistema contava il tempo in decimi di secondo, ma 0,1 in binario non è esatto. Dopo circa 100 ore l'errore accumulato spostò il calcolo e un missile non fu intercettato: 28 militari uccisi. È lo stesso fenomeno del capitolo 3.

Therac-25, 1985–87

Una macchina per radioterapia somministrò dosi letali per una *race condition*: due parti del software lavoravano in parallelo e lo stato della macchina diventava incoerente.

Millennium bug

Per risparmiare memoria molti programmi salvavano l'anno con due cifre: il 2000 diventava «00», cioè 1900. Fu evitato con anni di lavoro preventivo.

Anno 2038

Molti sistemi contano il tempo in secondi dal 1° gennaio 1970 con un intero a 32 bit, che si esaurisce il 19 gennaio 2038. I sistemi moderni usano 64 bit; il rischio resta nei dispositivi vecchi.

Nomi e origini

- **Python** prende il nome dai Monty Python, non dal serpente.
- **JavaScript** fu scritto in circa dieci giorni nel 1995; il nome richiamava Java solo per motivi commerciali.
- **Ada** porta il nome di Ada Lovelace, che nel 1843 descrisse un algoritmo per la macchina analitica di Babbage.
- **C** si chiama così perché viene dopo il linguaggio **B**.

- **Il primo «bug»**: nel 1947 il gruppo di Grace Hopper trovò una falena in un relè del calcolatore Harvard Mark II e la incollò sul registro. La parola esisteva già in ingegneria, ma l'episodio la rese celebre.
- **L'Apollo 11** atterrò con un software di guida sviluppato dal gruppo di Margaret Hamilton al MIT: durante la discesa il computer segnalò un sovraccarico e scartò i compiti meno importanti, come previsto dal progetto.

Stranezze che vengono dall'hardware o dalla storia

- **Da `0` o da 1?** I linguaggi della famiglia C contano da `0`; Fortran, MATLAB, R, Julia e Lua contano da 1, come gli ingegneri.
- **Excel e il 29 febbraio 1900.** Excel considera bisestile il `1900`, che non lo era: l'errore fu mantenuto per compatibilità con Lotus 1-2-3.
- **Excel e la genetica.** La conversione automatica in date trasformava nomi di geni come SEPT1 in date; nel `2020` quei geni sono stati rinominati.
- **Il «null» da un miliardo di dollari.** Tony Hoare, che introdusse il riferimento nullo nel 1965, l'ha definito il suo «errore da un miliardo di dollari». Linguaggi recenti come Rust, Kotlin e Swift lo limitano di proposito.
- **Turing-completezza.** Un linguaggio con memoria, condizioni e ripetizione può in teoria calcolare tutto ciò che è calcolabile. Lo sono anche Excel con LAMBDA e Dynamo: la differenza tra linguaggi è di comodità, non di potenza.

Fonti e approfondimenti

I fatti tecnici di questa guida sono stati controllati sulle fonti seguenti. Le pagine sono in inglese, salvo dove indicato.

- IETF, RFC 9562 · Universally Unique IDentifiers (UUIDs), maggio 2024: struttura, versioni e generazione degli UUID; sostituisce l'RFC 4122.
- IfcOpenShell, modulo ifcopenshell.guid: conversione tra UUID e GlobalId IFC a 22 caratteri, alfabeto usato da IFC.
- IfcOpenShell, documentazione: lettura e modifica di file IFC con Python.
- Autodesk, Revit API · Element.UniqueId: identificativo stabile degli elementi.
- Autodesk, Revit API Developer's Guide: transazioni, raccolta di elementi, unità interne.
- J. Tammik, UniqueId versus DWF and IFC GUID: differenza tra UniqueId, GUID di esportazione e GUID IFC.
- USB Implementers Forum, Human Interface Devices (HID): lo standard con cui tastiere e mouse comunicano con il computer.
- Microsoft, About Keyboard Input: come Windows trasforma la pressione di un tasto in un evento per l'applicazione.
- Python Software Foundation, Floating-Point Arithmetic: Issues and Limitations: perché 0,1 + 0,2 non fa 0,3.

COME È NATA QUESTA GUIDA

È uno dei «manuali di Claude»: l'ha scritta Claude, l'assistente di intelligenza artificiale di Anthropic, seguendo indicazioni precise su argomenti, struttura e livello di dettaglio. Le grafiche sono disegnate per questa guida. Segnalazioni e correzioni sono benvenute, dal sito.